

17. Batch Processing in JDBC

Batch processing is a feature in JDBC that allows you to execute a group of SQL statements as a single batch. This reduces the number of database calls, thus improving performance and efficiency, especially for repetitive tasks like inserting or updating multiple rows. Instead of executing a single query, we can execute a batch (group) of queries. It makes the performance fast. It is because when one sends multiple statements of SQL at once to the database, the communication overhead is reduced significantly, as one is not communicating with the database frequently, which in turn results to fast performance.

The `java.sql.Statement` and `java.sql.PreparedStatement` interfaces provide methods for batch processing.

17.1 Key Features of Batch Processing in JDBC

1. **Reduced Overhead:** Minimizes the number of network round-trips between the application and the database.
2. **Efficient Resource Usage:** Processes multiple statements together, reducing resource usage.
3. **Error Handling:** Allows you to handle errors gracefully with `BatchUpdateException`.

17.2 Methods of Statement interface

The required methods for batch processing are given below:

Method	Description
<code>void addBatch(String query)</code>	The <code>addBatch(String query)</code> method of the <code>CallableStatement</code> , <code>PreparedStatement</code> , and <code>Statement</code> is used to single statements to a batch.
<code>int[] executeBatch()</code>	The <code>executeBatch()</code> method begins the execution of all the grouped together statements. The method returns an integer array, and each of the element of the array represents the updated count for respective update statement.
<code>boolean DatabaseMetaData.supportsBatchUpdates() throws SQLException</code>	If the target database facilitates the batch update processing, then the method returns true.
<code>void clearBatch()</code>	The method removes all the statements that one has added using the <code>addBatch()</code> method.

Example to Check Batch Update Support:

```

package com.lokesh;

import java.sql.Connection;
import java.sql.DatabaseMetaData;
import java.sql.DriverManager;
import java.sql.SQLException;

public class CheckBatchUpdateSupport {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/organization";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = "root";

    public static void main(String[] args) {
        Connection connection = null;

        try {
            // 1. Establish connection
            connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);

            // 2. Get DatabaseMetaData object
            DatabaseMetaData metaData = connection.getMetaData();

            // 3. Check if batch updates are supported
            boolean isBatchSupported = metaData.supportsBatchUpdates();

            // 4. Display the result
            if (isBatchSupported) {
                System.out.println("Batch updates are supported by the JDBC driver.");
            } else {
                System.out.println("Batch updates are NOT supported by the JDBC driver.");
            }

        } catch (SQLException e) {
            e.printStackTrace();
        } finally {
            // 5. Close connection
            if (connection != null) {
                try {
                    connection.close();
                } catch (SQLException closeEx) {
                    closeEx.printStackTrace();
                }
            }
        }
    }
}

```

Output:

```
Batch updates are supported by the JDBC driver.
```

Example for Batch Processing with Statement interface:

```

package com.lokesh;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class BatchProcessingWithStatement {

```

```

private static final String DB_URL = "jdbc:mysql://localhost:3306/organization";
private static final String DB_USER = "root";
private static final String DB_PASSWORD = "root";

public static void main(String[] args) {
    Connection connection = null;
    Statement statement = null;

    try {
        // 1. Establish connection
        connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);

        // 2. Disable auto-commit mode
        connection.setAutoCommit(false);

        // 3. Create Statement object
        statement = connection.createStatement();

        // 4. Add SQL commands to the batch for `employees` table
        statement.addBatch("INSERT INTO company (name, department, salary) VALUES
                                ('John Doe', 'Engineering', 75000.00)");
        statement.addBatch("INSERT INTO company (name, department, salary) VALUES
                                ('Jane Smith', 'Marketing', 65000.00)");

        // 6. Execute the batch
        int[] updateCounts = statement.executeBatch();

        // 7. Commit the transaction
        connection.commit();

        System.out.println("Batch executed successfully. Rows affected: " +
                                updateCounts.length);

    } catch (SQLException e) {
        e.printStackTrace();

        // Rollback in case of an error
        if (connection != null) {
            try {
                connection.rollback();
            } catch (SQLException rollbackEx) {
                rollbackEx.printStackTrace();
            }
        }
    } finally {
        // Close resources
        try {
            if (statement != null) {
                statement.close();
            }
            if (connection != null) {
                connection.close();
            }
        } catch (SQLException closeEx) {
            closeEx.printStackTrace();
        }
    }
}

```

Output:

```
Batch executed successfully. Rows affected: 2
```

Example for Batch Processing with PreparedStatement interface:

```

package com.lokesh;

import java.math.BigDecimal;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class BatchProcessingWithPreparedStatement {

    private static final String DB_URL = "jdbc:mysql://localhost:3306/organization";
    private static final String DB_USER = "root";
    private static final String DB_PASSWORD = "root";

    public static void main(String[] args) {
        Connection connection = null;
        PreparedStatement preparedStatement = null;

        try {
            // 1. Establish connection
            connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);

            // 2. Disable auto-commit mode
            connection.setAutoCommit(false);

            // 3. Prepare the SQL statement
            String sql = "INSERT INTO company (name, department, salary) VALUES (?, ?, ?)";
            preparedStatement = connection.prepareStatement(sql);

            // 4. Add multiple SQL commands to the batch
            preparedStatement.setString(1, "John Doe");
            preparedStatement.setString(2, "Engineering");
            preparedStatement.setBigDecimal(3, new BigDecimal("75000.00"));
            preparedStatement.addBatch();

            preparedStatement.setString(1, "Jane Smith");
            preparedStatement.setString(2, "Marketing");
            preparedStatement.setBigDecimal(3, new BigDecimal("65000.00"));
            preparedStatement.addBatch();

            preparedStatement.setString(1, "Emily Johnson");
            preparedStatement.setString(2, "HR");
            preparedStatement.setBigDecimal(3, new BigDecimal("55000.00"));
            preparedStatement.addBatch();

            // 5. Execute the batch
            int[] updateCounts = preparedStatement.executeBatch();

            // 6. Commit the transaction
            connection.commit();

            System.out.println("Batch executed successfully. Rows affected: " +
                               updateCounts.length);

        } catch (SQLException e) {
            e.printStackTrace();

            // Rollback in case of an error
            if (connection != null) {
                try {
                    connection.rollback();
                } catch (SQLException rollbackEx) {
                    rollbackEx.printStackTrace();
                }
            }
        }
    }
}

```

```
    }  
  } finally {  
    // Close resources  
    try {  
      if (preparedStatement != null) {  
        preparedStatement.close();  
      }  
      if (connection != null) {  
        connection.close();  
      }  
    } catch (SQLException closeEx) {  
      closeEx.printStackTrace();  
    }  
  }  
}
```

Output:

Batch executed successfully. Rows affected: 3